

No-Restart Elasticity in an Adaptive Runtime System for Cloud-Native HPC

Aditya Bhosale

University of Illinois Urbana-Champaign
Urbana, IL
adityapb@illinois.edu

Laxmikant Kale

University of Illinois Urbana-Champaign
Urbana, IL
kale@illinois.edu

Abstract—Exploiting discounted spot instances for HPC requires an application to change its resource allocation at runtime, shrinking ahead of an interruption and expanding onto replacement capacity. Existing elasticity mechanisms implement rescaling as a full process teardown followed by a cold restart at the new processor count, and the restart stage accounts for up to 95% of the total overhead, growing with node count and increasing substantially on GPUs due to CUDA context initialization. In this paper, we present a no-restart rescaling mechanism for the Charm++ runtime system in which surviving processes never exit: a rescaling operation consists of a membership negotiation with a lightweight external coordinator, reconciliation of the UCX communication endpoints with the new cluster view, and a return to the top of the runtime initialization path that preserves live application state. This reduces the cost of a rescaling operation from multiple seconds to 8–15ms on CPUs and 7–11ms on GPUs, at 4 to 32 instances. Because processes survive, GPU device state persists in place, eliminating the checkpointing daemons previously required for GPU elasticity, and a launcher-independent bootstrap mechanism removes the dependence on supervised process managers, which are incompatible with spot instance interruptions. Integrated with an existing spot instance management framework, the mechanism cuts the end-to-end overhead of eight simultaneous interruptions to 0.2% of runtime on CPUs and 0.9% on GPUs, and raises the rate at which a job can be rescaled below 1% overhead by a factor of seven.

Index Terms—HPC, cloud, elasticity, spot instances, GPUs

I. INTRODUCTION

The convergence of HPC and cloud computing has accelerated in recent years. Cloud providers now offer HPC-oriented instance types and high-performance interconnects, while the capital cost of dedicated supercomputers and the growth of AI/ML workloads continue to drive HPC users toward cloud platforms. Among the strongest economic arguments for the cloud is the spot market: providers sell spare capacity at discounts of 60–90% under the condition that instances may be interrupted with only a short warning when demand for on-demand capacity surges. Prior work has shown that adaptive runtime systems such as Charm++ can exploit spot instances for tightly coupled HPC applications by rescaling the application in response to interruptions, shrinking it onto the surviving instances and expanding it again when replacement capacity arrives, achieving substantial cost savings [1], and that proactive replacement policies can hide most of the capacity loss associated with an interruption [2].

A fundamental inefficiency underlies this prior work: every rescaling operation is implemented as a restart. Whether the application state is checkpointed to a shared filesystem or to Linux shared memory, changing the number of processing elements (PEs) requires terminating every process in the job and launching a new set of processes at the new PE count, which then restore the saved state. Results from previous work [2] show that this restart stage dominates the total cost of a rescaling operation. Of an approximately 2.3s shrink operation on CPUs, roughly 2s is spent in process restart; on GPUs, the restart cost rises to 6–7s of approximately 7.5s total because every new process must reinitialize its CUDA context. The restart cost also grows with the number of nodes, since application launch time scales with job size. In contrast, checkpoint, load balancing, and restore, the stages of a rescaling operation that perform essential work, constitute only a small fraction of the total overhead.

The restart-based model imposes costs beyond wall-clock overhead. On GPUs, process termination destroys the CUDA context, so GPU-resident data must be moved out of the terminating process. Prior work required a per-GPU daemon process and CUDA Interprocess Communication (IPC) machinery solely to preserve device data across the restart [2], [3]. The multi-second overhead also constrains resource management policy: rescaling must be treated as a rare event to be batched and deferred rather than an operation that can be invoked freely.

Moreover, the restart model is the only model that conventional launchers permit. MPI-style launchers such as `mpirun` and `PRRTE` maintain a supervised daemon on every host, abort the entire job when any daemon disconnects, and provide no mechanism for adding or removing hosts from a running job. A full teardown followed by a new launch is the only membership change they support, and prior systems relied on exactly this relaunch to rescale. Eliminating the restart therefore requires eliminating the dependence on a supervised launcher as well.

In this paper, we eliminate the restart. We present a rescaling mechanism for Charm++ over UCX in which surviving processes never exit. A shrink or expand operation proceeds as follows: (1) PE 0 negotiates the new cluster membership with a lightweight external TCP coordinator; (2) departing processes exit cleanly; (3) every surviving process reconciles

its UCX endpoints with the new membership view and returns control to a point at the top of the runtime entry function, re-executing initialization in a mode that preserves live state; and (4) joining processes start as new processes but bypass `main()`, adopting runtime state from a broadcast. The application then resumes its iteration loop. The entire sequence completes in approximately 10 ms on our test configuration, a reduction of more than two orders of magnitude relative to the restart-based approach.

Realizing this model requires addressing a substantial systems problem: every piece of runtime state that was designed to be initialized exactly once per process lifetime must now survive an arbitrary number of reinitializations within a single process lifetime. Handler tables, reduction trees, location managers, load balancer synchronization, and timers all carry state that is either initialization-time state, which must be preserved rather than re-created, or topology-dependent state, which must be adjusted rather than preserved. We describe the lifecycle pattern we developed to manage this distinction and the mechanisms it requires.

This paper makes the following contributions:

- We present a no-restart shrink/expand mechanism for a message-driven parallel runtime system, based on co-ordinated membership change, in-place UCX endpoint reconciliation, and reinitialization of surviving processes without process termination. The mechanism reduces rescaling overhead from seconds to approximately 10 ms (Sections III and IV).
- We present a launcher-independent bootstrap mechanism built on a standalone membership coordinator and an SSH-based launcher that spawns the ranks and exits, leaving no supervising daemons. This removes the process manager dependence that makes conventional launchers incompatible with spot instance interruptions (Section III).
- We show that the no-restart model provides GPU elasticity without checkpointing daemons: because processes survive rescaling operations, CUDA contexts and device-resident data persist in place, making the CUDA IPC daemon subsystem required by prior work unnecessary (Section IV).
- We integrate the mechanism with an existing spot instance management framework with proactive capacity rebalancing, analyze how low-cost rescaling changes the space of viable interruption-handling policies, and evaluate the system end-to-end on CPU and GPU spot fleets (Sections V and VI).

II. BACKGROUND AND MOTIVATION

A. Charm++ and Migratable Objects

Charm++ [4], [5] is an asynchronous, message-driven parallel programming model in which computation is expressed in terms of migratable C++ objects called *chares*, mapped to processing elements (PEs) by the runtime system and communicating through non-blocking entry method invocations

delivered by a distributed location management system. Chares are organized into indexed collections; two special collection types, *groups* and *nodegroups*, have exactly one member per PE and per process, and the runtime system itself is largely built from groups, which implement per-PE services such as load balancing, location management, and reductions. Applications also declare *read-only* variables, global data initialized on PE 0 at startup and broadcast to all PEs. Because applications are overdecomposed into many more chares than PEs and the runtime manages object placement, it can migrate chares during execution, enabling measurement-based dynamic load balancing [5] and, most relevant to this work, elasticity (termed malleability in the job scheduling literature): changing the number of PEs available to a running application [6].

B. The Kill-and-Restart Elasticity Model

Prior implementations of shrink/expand in Charm++ follow a model that we refer to as kill-and-restart [1], [6]. To shrink, the runtime system first migrates chares away from the PEs to be removed using a load balancing step that also rebalances the remaining load. The application state, consisting of read-only global data, group (per-PE) objects, and all chare data, is then checkpointed into Linux shared memory, itself an optimization over the earlier practice of checkpointing to a shared filesystem, on the surviving hosts. Every process in the job exits, the launcher restarts the application with the new PE count, and the new processes restore their state from the shared-memory checkpoint and resume execution. Expansion is symmetric: the runtime checkpoints, restarts at the larger PE count, restores, and performs a load balancing step to populate the new PEs.

For GPU applications, this model requires additional machinery. Process termination destroys the CUDA context and, with it, all device allocations. To avoid staging device data through host memory on every rescaling operation, prior work launched a persistent daemon process for each GPU. At checkpoint time, each PE passed its device buffer pointers to the daemon using CUDA IPC, and the daemon copied the data into buffers that it owned. After restart, the daemon passed the data back to the application processes [2], [3]. This approach kept checkpoint and restore traffic on the device but introduced a daemon lifecycle that must be managed on every host, including replacement instances.

C. Spot Instances and CharmCloudManager

Cloud providers sell spare capacity as spot instances at significant discounts, with the caveat that instances can be interrupted after a short warning (two minutes on AWS EC2). CharmCloudManager [1] manages Charm++ applications on mixed on-demand and spot fleets: a monitoring task watches for interruption notices and newly launched instances, maintains the nodelist, and signals the application to shrink before an interruption takes effect and to expand when replacement capacity arrives. Prior work [2] added capacity rebalancing, launching a replacement proactively when an instance is flagged at elevated risk so that the application rescales

TABLE I
BREAKDOWN OF SHRINK OVERHEAD UNDER THE KILL-AND-RESTART
MODEL WHEN ONE INSTANCE IS INTERRUPTED, FROM [2] (JACOBI2D,
16,384 × 16,384 GRID, 16 INSTANCES).

Stage	CPU (c6gn.large)	GPU (g4dn.xlarge)
Checkpoint	~0.05 s	~0.05 s
Load balance	~0.15 s	~0.35 s
Restart	~1.9 s	~6.8 s
Restore	~0.05 s	~0.05 s
Total	~2.2 s	~7.2 s

once, directly onto the replacement, rather than shrinking on interruption and expanding later; this halved the interruption-handling overhead by eliminating one of the two restarts.

D. Motivation: Restart Dominates Rescaling Cost

Table I summarizes the stage-by-stage overhead of a shrink operation under the kill-and-restart model, as measured in prior work [2]. Two observations motivate the present work.

First, restart is the dominant cost. In-memory checkpointing already reduced the checkpoint and restore stages to tens of milliseconds, and the load balancing cost is modest. Process teardown and cold restart account for 85–95% of the total overhead. Moreover, unlike the other stages, the restart cost increases with job size, because application launch time grows with the number of nodes. On GPUs, the restart penalty approximately triples due to CUDA context initialization, which requires multiple seconds per process. Any further significant reduction in rescaling overhead must therefore come from the restart stage.

Second, the restart model is imposed by the launchers it runs on. `mpirun` and `PRRTE` maintain a supervised daemon on every host, treat the loss of any daemon as a fatal job event, and provide no mechanism for changing the host set of a running job. A full teardown followed by a relaunch is therefore the only membership change they support. Prior work rescaled through exactly this relaunch, which incidentally protected the job from launcher failures: every process, and the launcher session itself, terminated before any host disappeared. This protection is unavailable to a no-restart design, in which surviving processes must continue executing while interrupted hosts disappear, a condition that supervised launchers treat as fatal. Launcher independence must therefore be addressed as an explicit design requirement.

III. DESIGN

A. Overview

The central idea of our design is that a rescaling operation should be a reconfiguration of live processes rather than a restart of the job: PE 0 negotiates the new membership with an external TCP coordinator; departing PEs evacuate their chares through a load balancing step and exit; survivors reconcile their UCX endpoints with the new view and `longjmp` back to the runtime entry function, re-executing initialization in a

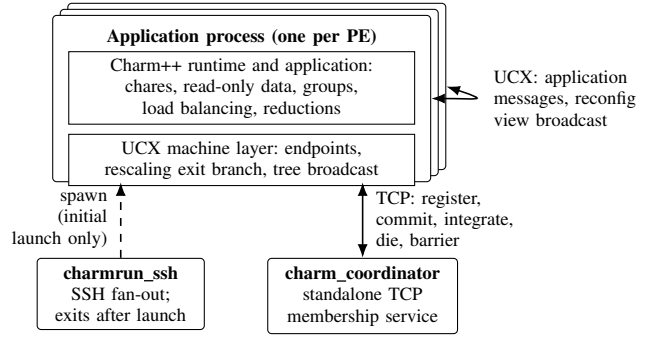


Fig. 1. System components. The coordinator is the only persistent external process. The launcher spawns the initial cluster and plays no further role in execution. Survivors learn reconfiguration views over UCX; only PE 0, removed members, and newcomers communicate with the coordinator during a rescaling operation.

mode that skips or adapts work that must not repeat (Section IV); newcomers bypass `main()` and adopt a broadcast of read-only and group state from PE 0; and a recorded resume callback re-enters the application. No process image is reloaded, no CUDA context is destroyed, and no state is written to a filesystem or shared memory.

Three kinds of processes participate in the system (Figure 1): the application binary, with one process per PE; a single standalone coordinator process that maintains the canonical cluster membership; and a launcher that spawns the initial cluster and subsequently plays no role. The coordinator is deliberately built without any Charm++ dependencies; it is a small TCP server that is independent of the runtime system.

B. The Coordinator

The coordinator maintains the canonical cluster *view*, consisting of an epoch number and an ordered member list in which each entry carries a node ID and the corresponding node’s UCX worker address. The epoch number is a monotonically increasing counter that identifies a version of the membership; it is incremented on every committed reconfiguration. Protocol messages carry the epoch of the view they refer to, which allows the coordinator and the ranks to detect requests and barrier arrivals that belong to an outdated view. The coordinator implements a small message protocol over TCP, organized into four message flows.

1) *Initial registration*: Each rank of the initial launch sends a `REGISTER_INITIAL` message containing its launcher-assigned node ID and its UCX address. Once the expected number of ranks has registered, the coordinator replies to all of them with the complete member list. This exchange provides the address distribution that a process management interface (PMI) [7], [8] would otherwise perform.

2) *Newcomer registration*: A process joining during an expansion sends a `REGISTER_NEWCOMER` message with its UCX address and receives an immediate snapshot of the current cluster. The process is then queued in FIFO order until a commit consumes it, at which point the coordinator sends it

an INTEGRATE message containing its assigned node ID and the final post-rescale view.

3) *Commit*: PE 0 drives every reconfiguration. It first issues a QUERY_PENDING message to determine how many newcomers are queued, and then sends a COMMIT message containing the epoch, the set of nodes to remove, and the number of newcomers to absorb. The coordinator compactly renumbers the survivors, appends the consumed newcomers, increments the epoch, sends a DIE message to each removed member and an INTEGRATE message to each consumed newcomer, and returns the new view to PE 0 in a COMMIT_REPLY message.

4) *Barrier*: The coordinator provides a barrier across all live ranks, used during job shutdown under coordinator bootstrap (Section III-C), where it substitutes for the barrier that PMI would otherwise provide at termination.

An important scalability property of the design is that survivors do not learn the new view over TCP. A straightforward alternative, in which the coordinator pushes the view to every survivor, serializes an $O(N)$ fan-out through a single TCP server. Instead, PE 0 serializes the view into a reconfiguration message and forwards it over UCX along a binary tree rooted at itself, each survivor applying the view delta and forwarding to its children. The closing synchronization follows the same principle, a barrier over the new endpoints along the same tree. The coordinator therefore communicates only with PE 0, the removed members, and the consumed newcomers, and its work is independent of the number of survivors.

C. Bootstrap Modes and Launcher Independence

The initialization routine of the UCX machine layer supports three entry paths.

1) *Survivor reinitialization*: After the `longjmp`, a survivor's machine layer initialization performs almost no work: it reports the new node count and the process's new node ID, both of which were recorded before the jump. The endpoints were already reconciled with the new view in the rescaling branch of the exit path, so communication is available as soon as initialization returns.

2) *Newcomer*: A process launched with the `+newcomer` flag initializes UCX locally, registers with the coordinator, and receives a snapshot of the current cluster. It then speculatively opens endpoints to the current members and blocks awaiting the INTEGRATE message; the endpoint handshakes complete while the application continues to run. When the final view arrives, the newcomer compares its speculative endpoints against the committed membership, creating, retaining, or closing endpoints as needed, and joins the survivors at a tree barrier.

3) *Initial launch*: Under coordinator bootstrap, the launcher passes each rank its node ID, the node count, and the coordinator address on the command line; ranks register with the coordinator and build endpoints directly from the returned member list, without involving a process management interface. For non-elastic runs, conventional launchers remain

supported, with PMI providing rank assignment and address exchange.

Coordinator bootstrap is what makes elastic execution possible. Supervised launchers such as `mpirun` treat any daemon disconnection as fatal: an interrupted spot host, or even a host that is cleanly vacated after a shrink, terminates the job. PR RTE has less aggressive daemon-loss semantics but still aborts on TCP connection close. Retaining PMI without the supervised daemons is not an option, because PMI is an interface to the process manager rather than an independent service: its server side is hosted by those daemons, which supply rank assignment and the key-value store [7], [8]. Abandoning the supervised daemon model therefore requires a different provider for both roles. The coordinator, which must exist anyway for membership management, assumes them. We therefore developed `charmrun_ssh`, a launcher that fans out over SSH, spawns the ranks, and exits. The disappearance of a vacated host has no effect on the surviving cluster; the only component that must remain reachable is the coordinator.

D. The Shrink Flow

A shrink is triggered by an external request. In our system, the resource manager delivers a keep/remove bitmap over the Charm++ Converse Client-Server (CCS) remote request interface to PE 0 (Section V). The request is folded into the next load balancing step: the load balancer computes a migration plan that evacuates all chares from the departing PEs while rebalancing load across the survivors, and the migrations execute normally. When migration completes, the runtime enters the rescaling path:

- 1) The load balancer migrates all chares from the departing PEs while rebalancing load across the survivors. The rescaling path begins once migration completes.
- 2) A broadcast marks every PE as participating in a rescaling operation. This flag, set on all PEs, serves as the sole indicator that gates rescaling behavior throughout the runtime (Section IV). A barrier then ensures that every PE has entered the rescaling path before the reconfiguration proceeds.
- 3) Each PE computes its post-rescale node ID from the bitmap and enters the runtime exit path, which detects the rescaling operation and branches away from process termination.
- 4) PE 0 queries the coordinator for pending newcomers (of which there are none for a pure shrink) and then commits the removal set. Departing PEs receive the DIE message, close their coordinator connections, and exit. The remaining survivors receive the new view through the UCX tree broadcast originating at PE 0.
- 5) Every survivor reconciles its endpoint table with the new view: endpoints between surviving pairs are retained and re-indexed under the new numbering (Section IV-C), and endpoints to removed members are closed. A pure shrink creates no endpoints. Each survivor then joins a tree barrier executed over the new endpoints; its completion guarantees that every rank has finished reconciliation

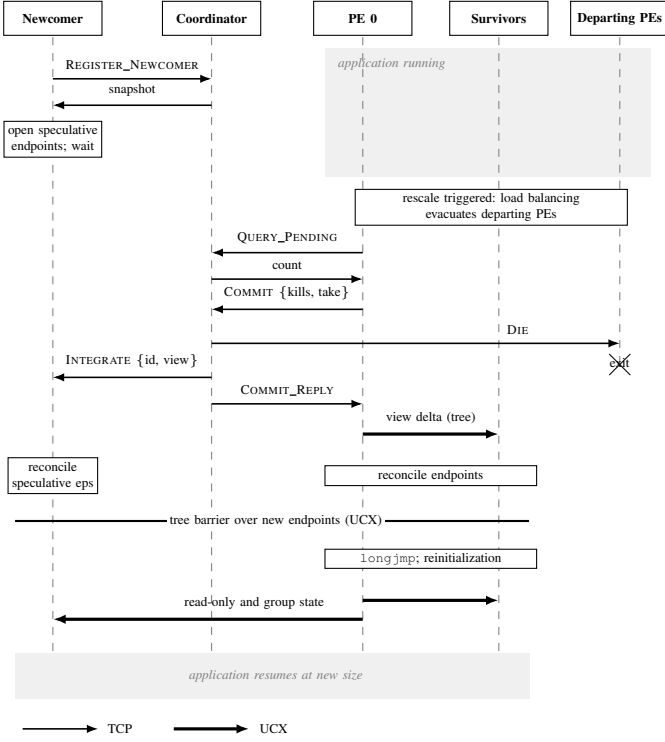


Fig. 2. Message sequence of a combined rescaling operation that removes the departing PEs and admits a newcomer in a single commit. Thin arrows are TCP messages to and from the coordinator; thick arrows are UCX messages. The coordinator exchanges messages only with PE 0, the departing PEs, and the newcomer; survivors receive the new view, the barrier, and the state broadcast over UCX. Newcomer registration and speculative endpoint setup overlap with normal application execution.

before any application traffic flows, and it constitutes the first exercise of the new endpoints.

- 6) Each survivor executes `longjmp` to the `setjmp` point in the runtime entry function. Runtime initialization executes again in survivor mode, PE 0 broadcasts read-only and group state, and the recorded resume callback re-enters the application.

E. The Expand Flow

Expansion shares the same structure with two additions (Figure 2). First, newcomers are spawned by the resource manager over SSH on the new hosts as soon as replacement capacity becomes available. They register with the coordinator, build speculative endpoints, and wait, all while the application continues to run. Second, when the rescaling operation is triggered, PE 0’s `QUERY_PENDING` message returns the number of queued newcomers, and its `COMMIT` message absorbs the minimum of the requested and queued counts. The coordinator’s `INTEGRATE` message delivers each newcomer its node ID and the final view. Each newcomer reconciles its speculative endpoints against the final view and joins the survivors at the tree barrier.

As in the kill-and-restart model, a newcomer does not execute `main()`: it adopts the read-only data and group state

that PE 0 broadcasts after its own `longjmp`. The post-expand resume callback triggers a load balancing step that populates the new PEs with migrated chares.

IV. STATE MANAGEMENT ACROSS REINITIALIZATION

While the rescaling mechanism of Section III is structurally simple, preserving the correctness of a mature runtime system across it constitutes the principal engineering challenge of this work. When a survivor transfers control back to the runtime entry function and re-executes runtime initialization, it runs an initialization path that was written under the assumption of executing exactly once per process lifetime. Every subsystem touched by that path holds state that falls into one of two categories, and the two categories require opposite treatments.

A. Two Categories of Runtime State

The first category is *initialization-time state* that must be preserved rather than re-created: handler tables, registered readonly, group, and chare metadata, CCS handler registrations, and timer epochs. Re-executing the code that initializes this state would leak, reorder, or destroy live registrations. The gate is a single global flag, set when `setjmp` returns a nonzero value, that is, on every rescaling restart. Initialization sites that create this category of state check the flag and skip their work.

The second category is *topology-dependent state* that must be adjusted rather than preserved: any state that encodes the PE count, a PE numbering, a spanning tree, or the progress of an in-flight collective operation. Preserving such state unmodified would leave survivors addressing removed PEs or waiting for contributions that can never arrive. Each such subsystem exposes a rescaling reset method, invoked on every PE from a single post-restore hook. This hook executes at the point where PE 0’s broadcast has restored the read-only data and group tables, so the full runtime is available but the application has not yet resumed.

The design must also maintain symmetry between survivors and newcomers. Message handlers are identified by small integer indices assigned in registration order, so survivor reinitialization must execute the same handler registration sequence as newcomer initialization. A gate that skips a registration on survivors but not on newcomers, or the reverse, silently misaligns every subsequent handler index between the two, and the first message dispatched across the mismatch corrupts the receiving process. This constraint determined where gates could be placed: registration sequences always execute, and only the mutation of already populated tables is skipped, with handler slot assignment made idempotent when a slot is already populated.

B. Treatment of Runtime States

Table II summarizes the required state treatments. We describe the most instructive cases in detail.

TABLE II
RUNTIME STATE REQUIRING EXPLICIT TREATMENT ACROSS A RESCALING OPERATION.

Subsystem	Treatment
Handler tables, CCS table	Preserve; skip reinitialization on survivors; idempotent slot assignment
Reduction manager (per group)	Reset in-flight state; preserve sequence numbers; rebase contribution counts on shrink; rebuild tree with explicit activation
Location manager	Clear location cache; recompute home PEs; re-key local records; re-announce locations to home PEs
Array element tables	Re-key per-array and per-PE element hash tables under re-encoded identifiers
Message dispatch	Buffer arrivals from faster-restoring peers during the restore window; dispatch after restore completes
Load balancer	Clear in-progress flags; drain buffered rescaling requests; reset migration counters; reset synchronization barrier; propagate barrier epoch to newcomers
Timers	Preserve high-resolution clock epoch across rescaling operations
Topology caches	Invalidate caches encoding the old PE count; skip hardware topology rediscovery
Exit-path flags, argument vector	Reset and rebuild on each rescaling operation

1) *Reduction trees*: The Charm++ reduction subsystem maintains, on each PE, a local contributor count, a count of expected child contributions, and a monotonically increasing reduction sequence number per contributor. Three distinct hazards arise. First, the sequence numbers represent application progress and must survive: flushing them, as a fresh boot would, desynchronizes survivors from the reduction sequence the application is executing. Second, the expected contribution count is itself distributed state: each PE carries a share of the global contributor count, and migration deliberately leaves this share in place when a contributor moves, with the total reconciled at the root. On a shrink, the departing PEs’ shares vanish with them while their former contributors, evacuated by the load balancing step, contribute from surviving PEs; each survivor must therefore rebase its share to its local contributor count, or the root observes more contributions than it expects and aborts. Third, when the reduction tree is rebuilt over the new PE numbering, a leaf with zero local contributors, typically an expansion newcomer that has not yet received migrated chares, must nevertheless actively inform its new parent that it has no outstanding contributions. Without this notification, the parent waits indefinitely and every subsequent reduction stalls.

2) *Location management*: The location manager maps chare indices to home PEs and caches known locations. Both are functions of the PE count and numbering, so after a rescaling operation a survivor’s tables are incorrect in every entry: cached locations refer to removed or renumbered PEs, and hashed records are keyed under stale identifier encodings. The reset clears the location cache, recomputes each local chare’s home PE under the new numbering, re-keys the local records, and re-announces locations to the new home PEs. Because survivors resume independently, a location request can reach a home PE before the corresponding announcement does; the home therefore buffers requests for identifiers it does not yet know and replies once the announcement arrives, rather

than dropping the request and leaving the requester’s messages buffered forever. Omitting any of these steps produces sends to removed endpoints or, more subtly, messages buffered indefinitely while awaiting an element that is present but keyed differently.

3) *Message dispatch during the restore window*: Survivors restore at different rates, and no barrier separates the end of one survivor’s restore from another’s. A fast survivor therefore begins sending post-rescale traffic, especially the location announcements and requests described above, while slower survivors are still rebuilding their state. The shutdown path routes incoming messages to a discard handler, which is correct under kill-and-restart, where a message arriving during teardown belongs to the terminated execution, but silently destroys valid post-rescale traffic under the no-restart model. The solution reuses the runtime’s own boot machinery: normal startup already buffers messages that arrive before initialization completes and dispatches them once it finishes, so the re-entry point installs the same buffering handler in place of the discard handler, and the post-restore hook drains the buffer before the application resumes. The transport re-initialization performs a barrier over the new membership immediately before control returns to the runtime entry point, so every survivor has installed the buffering handler before the first post-rescale message can be transmitted.

4) *Load balancer synchronization*: The load balancer’s synchronization barrier and central strategy carry per-step state: an in-progress flag, migration completion counters, and a barrier epoch. The rescaling path exits in the middle of a load balancing step, and the `longjmp` bypasses the code that would normally complete the barrier, so survivors would otherwise resume with a permanently disabled barrier or with counters that either satisfy the next step’s completion test spuriously or can never satisfy it. All of this state is reset in the post-restore hook. The barrier epoch must additionally be propagated to expansion newcomers: a newcomer starting at epoch zero while survivors are at epoch k has its barrier contributions discarded as stale, which causes the cluster to hang on the second load balancing step after the expansion. Newcomers therefore receive the current epoch as part of their integration.

C. Message-Layer Identity

UCX endpoints do not carry sender identity. Charm++ encodes the source node ID in the upper bits of each message tag, with the message type in the lower bits. Receive buffers are posted with a tag mask that matches only the message-type bits, so a posted receive accepts messages from any sender, and the receiver recovers the source from the matched tag. This sender-agnostic matching makes the receive path invariant under renumbering: receive buffers posted before a rescaling operation remain valid after it and are left in place. Only senders must adopt the new numbering, and in-flight traffic is drained before the reconfiguration, so no message carrying an old source ID crosses a rescaling operation; traffic sent after it that arrives while a survivor is still restoring is buffered rather

than dispatched (Section IV-B3). The compact renumbering itself is computed identically on every PE: the removal set is broadcast, and each PE derives its own new identifier, and those of all other PEs, deterministically.

D. GPU State Preservation

Under the kill-and-restart model, GPU elasticity required a per-GPU daemon that held device buffers across the process boundary using CUDA IPC (Section II-B). Under the no-restart model, this subsystem is unnecessary and has been removed. A survivor’s CUDA context, device allocations, streams, and registered host memory all persist unmodified across the `longjmp`; from the perspective of the device, the rescaling operation is an ordinary pause in kernel submissions. Device data moves only when the load balancer migrates a chore between PEs, in which case the existing Charm++ GPU messaging path applies, using GPUDirect RDMA where supported and host staging otherwise. Newcomer processes initialize CUDA once at spawn time, off the critical path, while awaiting integration.

Beyond the reduction in wall-clock overhead (Section VI), the reduction in system complexity is substantial: there is no daemon lifecycle to manage on each host, no daemon launch on replacement instances, no IPC handle exchange, and no failure modes arising from version or timing skew between the daemon and the application.

V. SPOT INSTANCE INTEGRATION

A. CharmCloudManager with No-Restart Rescaling

We integrate the no-restart mechanism into `CharmCloudManager` [1], [2]. The overall structure of the framework is unchanged: an execution task creates the EC2 fleet and launches the application, and a monitoring task watches for rebalance recommendations, interruption notices, and newly launched instances. As in prior work, the fleet mixes on-demand and spot instances. We assume throughout that at least one on-demand instance is present and that PE 0 resides on it, so that the rank driving every reconfiguration, and the coordinator alongside it, are never themselves interrupted; only spot instances are candidates for removal. The rescaling signal path, however, is new. Where the restart-based integration rewrote the nodelist and invoked `charmrun` again at the new PE count, the manager now operates as follows:

- 1) It launches the coordinator alongside the job when the fleet is created.
- 2) When new capacity becomes available, it connects to each replacement instance over SSH and spawns the application binary with the `+newcomer` flag and the coordinator address. The newcomers register with the coordinator, initialize CUDA in the case of GPU jobs, build speculative endpoints, and wait, while the application continues to run at full capacity.
- 3) When it decides to rescale, it delivers a keep/remove bitmap to PE 0 over CCS. At the next load balancing

step, the runtime evacuates the departing PEs and executes the reconfiguration described in Section III, at a cost of approximately 10 ms.

- 4) It terminates vacated instances after their PEs have exited.

Launcher independence (Section III-C) is what makes the final step safe: with `charmrun_ssh` and coordinator bootstrap, no supervised daemon runs on the vacated host, so its disappearance is not interpreted as a job failure. The initial launch, every expansion, and every shrink use the same binary and the same coordinator; no component of the job is ever relaunched.

Removals and additions can travel in the same COMMIT message, so an interruption for which replacement capacity is available is handled by a single rescaling operation combining the shrink and the expansion.

B. Capacity Rebalancing with Low-Cost Rescaling

We retain the proactive replacement policy of prior work [2]: upon a rebalance recommendation, the EC2 signal indicating elevated interruption risk, the fleet launches replacement capacity, and the manager defers the rescaling operation until all at-risk instances have been replaced, an interruption notice forces an emergency rescale, or a timeout expires. The policy exists to consolidate replacements into few rescaling operations.

Low-cost rescaling changes this design space in two ways. First, the motivation for consolidation weakens: the consolidation window traded idle replacement instances, paid for but unused, against additional multi-second rescaling operations, and at approximately 10 ms per operation, rescaling once per replacement as soon as it is ready becomes practical, eliminating idle time while maintaining target capacity. Second, the emergency path can act later. Under kill-and-restart the shrink itself consumed seconds of the two-minute warning window, forcing the manager to act early; with the mechanism cost negligible, the manager can wait late in the window for replacements to arrive, increasing the probability that an interruption is absorbed as a single combined operation rather than a shrink followed by a later expansion. In both directions the rescaling schedule becomes governed by capacity alone: rescale as soon as a replacement is ready, and wait when none is. We leave the exploration of both policy refinements to future work.

VI. EVALUATION

We evaluate the system along four axes: (1) a microbenchmark decomposition of the no-restart rescaling operation; (2) a comparison against the three interruption-handling modes measured in prior work [2]; (3) end-to-end application runs on spot fleets with injected interruptions; and (4) a rescaling frequency study that examines the policy space enabled by low-cost rescaling.

Experimental setup. We use the UCX machine layer of Charm++ with the no-restart extensions (the `ucx-linux-arm8` build on the CPU fleets and the

ucx-linux-x86_64-cuda build on the GPU fleets). CPU experiments use EC2 fleets of c6gn.large instances (two vCPUs each) running Jacobi2D, a communication-intensive five-point stencil benchmark; GPU experiments use g4dn.xlarge instances (one NVIDIA T4 each) running a CUDA implementation of the same computational pattern whose chare state resides in device memory. The fleets match those of prior work [2], both benchmarks run a $16,384 \times 16,384$ grid decomposed into $2,048 \times 2,048$ blocks, and both use the GreedyRefine load balancing strategy. Rescaling requests are delivered to PE 0 over CCS by the fleet manager of Section V, and spot interruptions are injected with the AWS Fault Injection Simulator.

A. Rescaling Overhead Breakdown

Figure 3 decomposes the rescaling operation and shows how it scales on the CPU and GPU fleets, shrinking by one instance and expanding back onto it at 4 to 32 instances. Endpoint closes are issued without blocking and complete during the post-rescale tree barrier, so the two are reported as one stage; the coordinator commit is a single round trip on PE 0, which, by the design of Section III-B, is independent of the number of survivors. On CPUs, total time grows from 8.3 ms to 14.5 ms for a shrink and from 5.2 ms to 11.7 ms for an expansion across an eightfold increase in fleet size: each doubling adds one to three milliseconds rather than multiplying the cost. The composition differs between the two operations. A shrink is dominated at every scale by endpoint reconciliation, which is nearly flat (5.0 to 6.2 ms) because the number of endpoints to close depends on the membership change, not the fleet size. An expansion instead is dominated by runtime initialization (2.9 to 5.4 ms), which the newcomer must execute in full while survivors skip it, and pays roughly 2 ms of endpoint work since it only adds endpoints. Orchestration remains below 0.35 ms throughout.

On GPUs, running the Jacobi2D benchmark, a shrink costs 7.3 ms at 4 instances and 11.1 ms at 32, an expansion 5.5 ms and 8.3 ms respectively, with similar stage decomposition. This is the central consequence of the no-restart design for accelerated workloads. Under kill-and-restart every process is replaced, so each rescaling operation pays CUDA context reinitialization on every surviving GPU, which cost 7 to 8 s per operation and motivated the checkpoint daemon of prior work (Section IV-D). Here survivors never lose their context, and the only GPU-specific cost, context creation on a newcomer, is overlapped with the newcomer’s wait for the membership commit. That overlap is essential: without it the context creation falls on the critical path of every survivor’s restore, and an expansion costs 172 ms at four instances instead of 5.5 ms. The result is a three orders of magnitude reduction, and GPU elasticity that is no more expensive than CPU elasticity.

B. Comparison with Restart-Based Handling

We compare four modes of handling a spot interruption, the first three of which were measured in prior work [2]: (A) checkpoint/restart through a shared filesystem (AWS EFS);

(B) in-memory kill-and-restart without capacity rebalancing; (C) in-memory kill-and-restart with capacity rebalancing; and (D) the mechanism presented in this paper.

Figure 4 shows the comparison. On CPUs, mode A costs 11–23 s depending on scale, mode B costs approximately 4.5–5.5 s (a shrink followed by an expansion), and mode C costs approximately 2.2–2.9 s (one combined rescaling operation). Mode D performs the same single combined operation in 8–15 ms plus the load balancing time, a reduction of more than two orders of magnitude in the mechanism cost relative to mode C. What remains of the interruption-handling overhead in mode D is load balancing, which is common to modes B–D and independent of the rescaling mechanism; total handling time is $3.5\times$ lower than mode C at four instances and $7.6\times$ lower at 32, and the rescaling component itself is no longer visible at the scale of the figure.

The GPU comparison is more favourable, because the restart term that dominates modes A–C is larger there: CUDA context reinitialization in every replaced process costs 6.4–7.4 s in mode C and 12.8–14.7 s in modes A and B. Mode D removes that term entirely rather than reducing it, since survivors never leave their process and the newcomer’s context creation is overlapped with its wait for the membership commit. A GPU rescaling operation costs 7–11 ms, and total interruption handling is $3.9\times$ lower than mode C at four instances and $13.5\times$ lower at 32, the advantage growing with scale as the load balancing term shrinks.

Two structural differences merit attention beyond the aggregate numbers. First, in modes A–C the dominant cost, restart, grows with job size, whereas in mode D the dominant cost, endpoint reconciliation, grows only with the size of the membership change, and the remaining costs are constant. Second, mode D eliminates the GPU checkpoint daemon entirely: the device-side checkpoint and restore stages of the prior GPU pipeline do not exist in this design, nor does the operational requirement of running daemon processes on every host.

C. End-to-End Runs with Injected Interruptions

Under the kill-and-restart system, eight simultaneous interruptions increased end-to-end runtime by approximately 16% on CPUs and 20% on GPUs without rebalancing, and by less than 1% and 3.4% respectively with rebalancing [2]; the residual overhead in the rebalancing case derives almost entirely from the single remaining restart. We repeat this experiment under the no-restart mechanism, with each batch of simultaneous interruptions handled by a single consolidated rescaling operation. Figure 5 shows the overhead each mode adds to an uninterrupted run. For eight simultaneous interruptions, the overhead falls from 2.9 s to 0.9 s on CPUs (0.6% to 0.2% of runtime) and from 13.2 s to 3.3 s on GPUs (3.4% to 0.9%). Immediate per-replacement rescaling, which trades up to $2k$ rescaling operations for k interruptions against the elimination of replacement instance idle time, becomes viable at this operating point; evaluating it requires interruptions

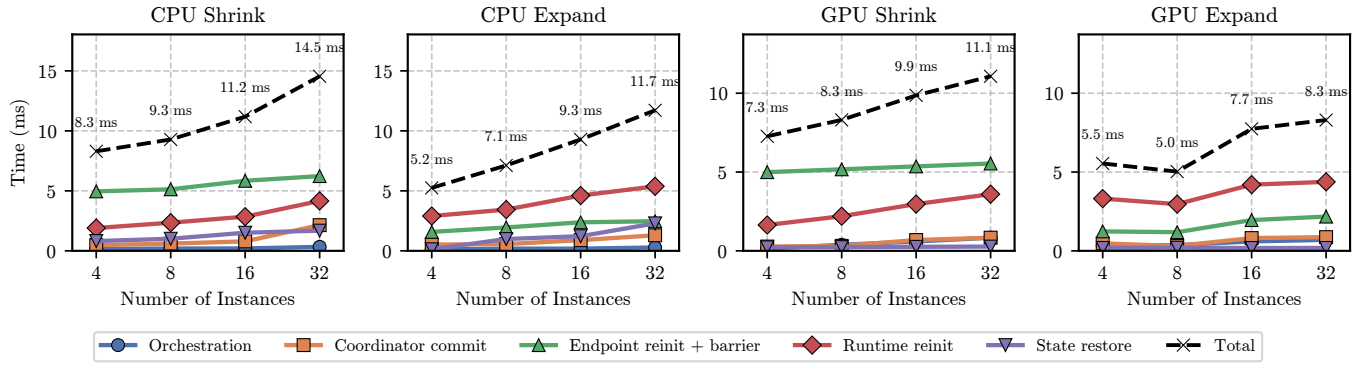


Fig. 3. No-restart rescaling time versus instance count, shrinking by one instance and expanding back, on c6gn.large fleets (2 PEs per instance) and g4dn.xlarge fleets (one NVIDIA T4 per instance).

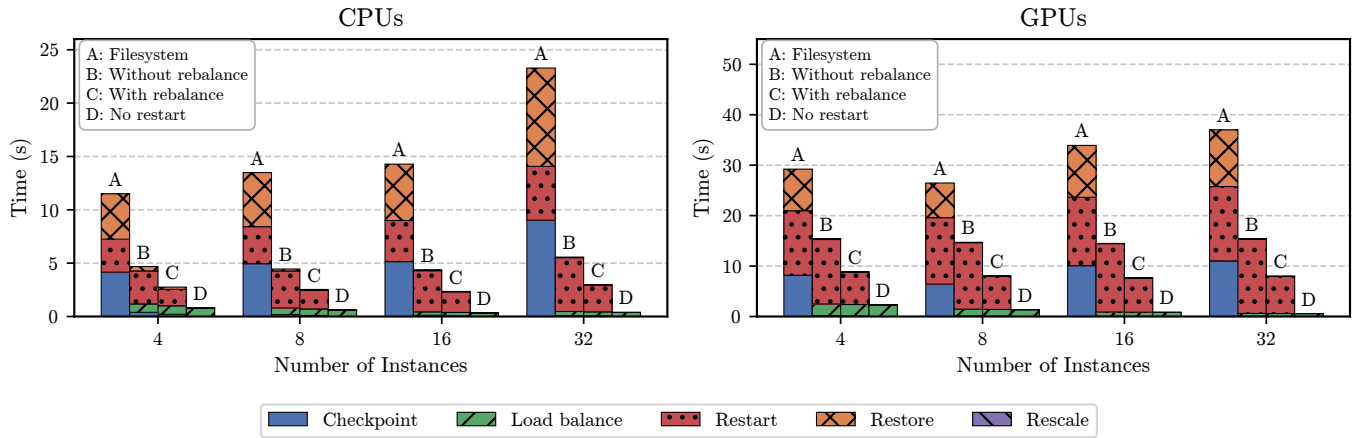


Fig. 4. Overhead of handling one spot interruption versus instance count, for the four handling modes, on CPU and GPU fleets. Modes A–C are the measurements of prior work [2].

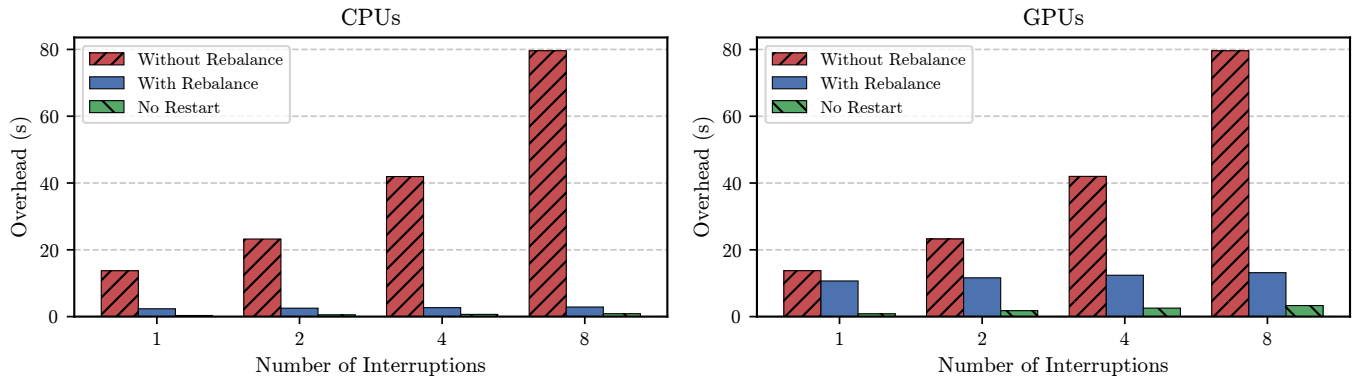


Fig. 5. End-to-end overhead over an uninterrupted run versus number of simultaneous spot interruptions, on 16 instances, for kill-and-restart handling without and with capacity rebalancing [2] and for the no-restart mechanism.

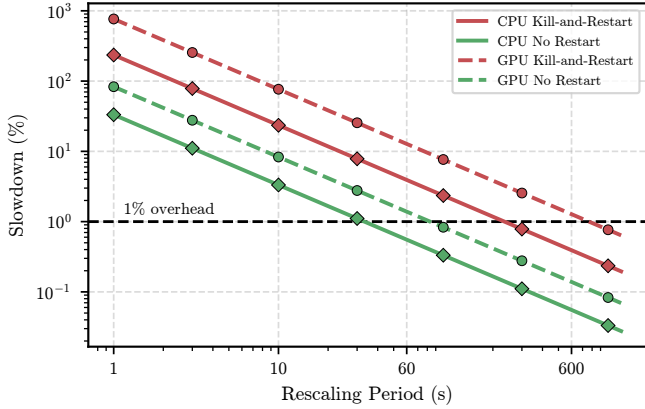


Fig. 6. Aggregate slowdown versus rescaling period on 16 instances.

staggered in time rather than simultaneous, and we leave it to future work.

D. Rescaling Frequency Study

Finally, we examine how frequently a job can be rescaled before the overhead becomes significant. We run Jacobi2D on CPUs and GPUs at a fixed problem size on 16 instances and force a rescaling operation, at a fixed period, sweeping the period, and measure the aggregate slowdown relative to a run with no rescaling.

Figure 6 shows the resulting slowdown, and the 1% line makes the comparison concrete: to keep overhead below 1%, kill-and-restart can sustain one rescaling operation every four minutes on CPUs and one every thirteen minutes on GPUs, whereas the no-restart mechanism sustains one every 33 seconds on CPUs and one every 83 seconds on GPUs, a 7–9× increase in sustainable frequency.

VII. RELATED WORK

Several papers have studied the performance of MPI on cloud platforms and on cloud interconnects such as AWS EFA [9]–[11], and the usability of cloud platforms for HPC proxy applications has been evaluated across providers [12]. Work targeting spot instances has largely relied on checkpoint/restart to survive interruptions [13], [14], including cost-aware checkpoint scheduling, or on replicated execution and hybrids of the two approaches [15]. Wu et al. [16] propose scheduling policies that exploit spot instances while meeting deadlines, achieving up to 84% cost savings. Bhosale et al. ran Charm++ applications on CPU spot fleets using in-memory kill-and-restart rescaling [1] and extended the approach to GPUs with proactive capacity rebalancing [2]. The present work removes the restart stage that dominated the remaining overhead in both.

Experimental elastic job support in Charm++ dates to Gupta et al. [6], which introduced the kill-and-restart model that this paper replaces. In the MPI ecosystem, elasticity efforts include DMR and related frameworks layered over process managers [17], the performance-driven reconfiguration

of Flex-MPI [18], and API extensions that grow and shrink the world communicator within adaptation windows [19]. MPI Sessions [20] relaxes the static-world assumption of the MPI standard, and recent work builds dynamic resource management on Sessions and PMIx [21]. However, production MPI reconfiguration continues to require either a job restart or the cooperation of a supervised process manager, which is precisely the component that spot interruptions disrupt. Elastic execution has also been studied in Kubernetes-based schedulers for HPC and machine learning workloads [22]–[25], which rescale at the granularity of pods and rely on application-level checkpoint/restart.

The closest technical relatives of our reinitialization approach come from MPI fault tolerance. Fenix [26], built on the failure-repair primitives of ULFM [27], and Reinit++ [28] implement global-restart recovery in which surviving processes are not terminated: after a failure, execution returns to an initialization point in each live process, the MPI runtime is repaired or reinitialized, and application state is rolled back from checkpoints. These systems establish the value of avoiding job redeployment, but they recover a job of unchanged size, roll back state rather than preserving it, and operate on the process manager’s supervised infrastructure. The no-restart mechanism extends the surviving-process model from failure recovery to deliberate reconfiguration: membership, numbering, and topology-dependent state change; live state is preserved without rollback; and no supervised infrastructure is required. To our knowledge, no prior tightly coupled HPC runtime system performs a rescaling operation without restarting the surviving processes.

VIII. CONCLUSION

This paper presented a no-restart rescaling mechanism for the Charm++ runtime system, in which surviving processes reconfigure in place rather than terminating and restarting at the new processor count. A rescaling operation costs 8 to 15 ms on CPU fleets and 7 to 11 ms on GPU fleets between 4 and 32 instances, two to three orders of magnitude below the multi-second cost of restart-based implementations, and grows by one to three milliseconds per doubling of the fleet. Because processes survive, device state persists in place and the checkpointing daemons previously required for GPU elasticity are eliminated. Integrated with a spot instance management framework, the mechanism reduces the end-to-end overhead of eight simultaneous interruptions from 0.6% to 0.2% of runtime on CPUs and from 3.4% to 0.9% on GPUs, and increases the rate at which a job can be rescaled below 1% overhead by a factor of seven on CPUs and nine on GPUs.

ACKNOWLEDGMENTS

Claude Fable was used to generate and to vet the logic of Figures 1 and 2, which illustrate the system components and the rescaling protocol of Section III.

REFERENCES

- [1] A. Bhosale, L. Kale, and S. Kokkila-Schumacher, "Efficient and Cost-Effective HPC on the Cloud," in *The 34th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '25)*. New York, NY, USA: ACM, jul 2025. [Online]. Available: <https://doi.org/10.1145/3731545.3744667>
- [2] A. Bhosale, A. Tahilyani, L. Kale, and S. Kokkila-Schumacher, "Towards an adaptive runtime system for cloud-native HPC," in *Proceedings of the Workshop on Advances in High-Performance Elastic and Scalable Systems (ASHES '26)*, 2026.
- [3] A. Bhosale and L. Kale, "An adaptive distributed stencil abstraction for gpus," in *2025 Supercomputing India (SCI)*, 2025, pp. 1–8.
- [4] L. Kalé and S. Krishnan, "CHARM++: A Portable Concurrent Object Oriented System Based on C++," in *Proceedings of OOPSLA'93*, A. Paepcke, Ed. ACM Press, September 1993, pp. 91–108.
- [5] B. Acun, A. Gupta, N. Jain, A. Langer, H. Menon, E. Mikida, X. Ni, M. P. Robson, Y. Sun, E. Totonii, L. Wesolowski, and L. V. Kalé, "Parallel programming with migratable objects: Charm++ in practice," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '14)*. New Orleans, LA, USA: IEEE, 2014, pp. 647–658.
- [6] A. Gupta, B. Acun, O. Sarood, and L. V. Kale, "Towards Realizing the Potential of Malleable Parallel Jobs," in *Proceedings of the IEEE International Conference on High Performance Computing*, ser. HiPC '14, Goa, India, December 2014.
- [7] P. Balaji, D. Buntinas, D. Goodell, W. Gropp, J. Krishna, E. Lusk, and R. Thakur, "PMI: A scalable parallel process-management interface for extreme-scale systems," in *Recent Advances in the Message Passing Interface (EuroMPI 2010)*, ser. LNCS, vol. 6305. Springer, 2010, pp. 31–41.
- [8] R. H. Castain, J. Hursey, A. Bouteiller, and D. Solt, "PMIx: Process management for exascale environments," *Parallel Computing*, vol. 79, pp. 9–29, 2018.
- [9] S. Chakraborty, S. Xu, H. Subramoni, and D. Panda, "Designing scalable and high-performance MPI libraries on Amazon elastic fabric adapter," in *2019 IEEE Symposium on High-Performance Interconnects (HOTI)*, 2019, pp. 40–44.
- [10] S. Xu, S. M. Ghazimirsaeed, J. M. Hashmi, H. Subramoni, and D. K. Panda, "MPI meets cloud: Case study with Amazon EC2 and Microsoft Azure," in *2020 IEEE/ACM Fourth Annual Workshop on Emerging Parallel and Distributed Runtime Systems and Middleware (IPDRM)*, 2020, pp. 41–48.
- [11] S. Xu, A. Shafi, H. Subramoni, and D. K. Panda, "ARM meets cloud: A case study of MPI library performance on AWS ARM-based HPC cloud with elastic fabric adapter," in *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2022, pp. 449–456.
- [12] V. Sochat, D. Milroy, A. Sarkar, A. Marathe, and T. Patki, "Usability evaluation of cloud for HPC applications," in *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25)*. New York, NY, USA: ACM, 2025, pp. 135–150.
- [13] S. Yi, D. Kondo, and A. Andrzejak, "Reducing costs of spot instances via checkpointing in the Amazon Elastic Compute Cloud," in *2010 IEEE 3rd International Conference on Cloud Computing*, 2010, pp. 236–243.
- [14] S. Yi, A. Andrzejak, and D. Kondo, "Monetary cost-aware checkpointing and migration on Amazon cloud spot instances," *IEEE Transactions on Services Computing*, vol. 5, no. 4, pp. 512–524, 2012.
- [15] Y. Gong, B. He, and A. C. Zhou, "Monetary cost optimizations for MPI-based HPC applications on Amazon clouds: Checkpoints and replicated execution," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*. New York, NY, USA: ACM, 2015.
- [16] Z. Wu, W.-L. Chiang, Z. Mao, Z. Yang, E. Friedman, S. Shenker, and I. Stoica, "Can't be late: Optimizing spot instance savings under deadlines," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. Santa Clara, CA: USENIX Association, Apr. 2024, pp. 185–203.
- [17] S. Iserte, R. Mayo, E. S. Quintana-Ortí, and A. J. P. na, "DMRlib: Easy-coding and efficient resource management for job malleability," *IEEE Transactions on Computers*, vol. 70, no. 9, pp. 1443–1457, 2021.
- [18] G. Martín, D. E. Singh, M.-C. Marinescu, and J. Carretero, "Enhancing the performance of malleable MPI applications by using performance-aware dynamic reconfiguration," *Parallel Computing*, vol. 46, pp. 60–77, 2015.
- [19] I. A. Comprés Ureña, A. Mo-Hellenbrand, M. Gerndt, and H.-J. Bungartz, "Infrastructure and API extensions for elastic execution of MPI applications," in *Proceedings of the 23rd European MPI Users' Group Meeting (EuroMPI 2016)*. ACM, 2016, pp. 82–97.
- [20] D. Holmes, K. Mohror, R. E. Grant, A. Skjellum, M. Schulz, W. Bland, and J. M. Squyres, "MPI sessions: Leveraging runtime infrastructure to increase scalability of applications at exascale," in *Proceedings of the 23rd European MPI Users' Group Meeting (EuroMPI)*. ACM, 2016, pp. 121–129.
- [21] D. Huber, M. Streubel, I. Comprés, M. Schulz, M. Schreiber, and H. Pritchard, "Towards dynamic resource management with MPI sessions and PMIx," in *Proceedings of the 29th European MPI Users' Group Meeting*, ser. EuroMPI/USA '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 57–67. [Online]. Available: <https://doi.org/10.1145/3555819.3555856>
- [22] D. J. Milroy, C. Misale, G. Georgakoudis, T. Elengikal, A. Sarkar, M. Drocco, T. Patki, J.-S. Yeom, C. E. A. Gutierrez, D. H. Ahn, and Y. Park, "One step closer to converged computing: Achieving scalability with cloud-native HPC," in *2022 IEEE/ACM 4th International Workshop on Containers and New Orchestration Paradigms for Isolated Environments in HPC (CANOPIE-HPC)*, 2022, pp. 57–70.
- [23] C. Misale, M. Drocco, D. J. Milroy, C. E. A. Gutierrez, S. Herbein, D. H. Ahn, and Y. Park, "It's a scheduling affair: GROMACS in the cloud with the KubeFlux scheduler," in *2021 3rd International Workshop on Containers and New Orchestration Paradigms for Isolated Environments in HPC (CANOPIE-HPC)*, 2021, pp. 10–16.
- [24] T.-T. Hsieh and C.-R. Lee, "Voda: A GPU scheduling platform for elastic deep learning in Kubernetes clusters," in *2023 IEEE International Conference on Cloud Engineering (IC2E)*, 2023, pp. 131–140.
- [25] A. Bhosale, K. Chandrasekar, L. Kale, and S. Kokkila-Schumacher, "An elastic job scheduler for hpc applications on the cloud," in *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25)*. New York, NY, USA: ACM, Nov. 2025, p. 12. [Online]. Available: <https://doi.org/10.1145/3731599.3767358>
- [26] M. Gamell, D. S. Katz, H. Kolla, J. Chen, S. Klasky, and M. Parashar, "Exploring automatic, online failure recovery for scientific applications at extreme scales," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '14)*. New Orleans, LA, USA: IEEE, 2014, pp. 895–906.
- [27] W. Bland, A. Bouteiller, T. Herault, G. Bosilca, and J. Dongarra, "Post-failure recovery of MPI communication capability: Design and rationale," *International Journal of High Performance Computing Applications*, vol. 27, no. 3, pp. 244–254, 2013.
- [28] G. Georgakoudis, L. Guo, and I. Laguna, "Reinit++: Evaluating the performance of global-restart recovery methods for MPI fault tolerance," in *High Performance Computing (ISC High Performance 2020)*, ser. LNCS, vol. 12151. Springer, 2020, pp. 536–554.

APPENDIX

ARTIFACT DESCRIPTION

PART 1: OVERVIEW OF CONTRIBUTIONS AND ARTIFACTS

A. Paper’s Main Contributions

- C_1 A no-restart shrink/expand mechanism for a message-driven parallel runtime system, in which surviving processes reconfigure in place through coordinated membership change, in-place endpoint reconciliation, and runtime reinitialization without process exit.
- C_2 A treatment of runtime state across reinitialization that separates state which must be preserved from state which must be adjusted, including the message buffering required while survivors restore at different rates.
- C_3 A launcher-independent bootstrap built on a standalone membership coordinator and an SSH launcher, removing the dependence on supervised process managers that spot interruptions disrupt.
- C_4 GPU elasticity without checkpointing daemons: because processes survive, CUDA contexts and device-resident data persist in place.
- C_5 Integration with a spot instance management framework and an analysis of how low-cost rescaling changes viable interruption-handling policy.

B. Computational Artifacts

Artifact	Contributions	Paper elements
A_1 : Rescaling breakdown	C_1, C_2, C_4	Figures 3, 4
A_2 : Interruption runs	C_1, C_4, C_5	Figure 5
A_3 : Frequency study	C_1, C_5	Figure 6

All artifacts run on the extended Charm++ runtime driven by the charm-aws fleet management and experiment harness (branch `canopie26` of both). The charm-aws README documents the machine images and build configurations, the AWS resources and quotas assumed, the exact command line for every experiment, and the figure generation steps:

- <https://github.com/charmplusplus/charm-aws/tree/canopie26>
- <https://github.com/charmplusplus/charm/tree/canopie26>

Every driver accepts `--gpu`, which selects the GPU machine image, instance type, benchmark, and Charm++ build, so the CPU and GPU variants of each experiment are the same code path. Every driver also accepts `--parse-only`, which re-derives the reported numbers from captured application output without allocating cloud resources, so the analysis and figures can be reproduced from an archived results set alone.

PART 2: ARTIFACT IDENTIFICATION

Artifact A_1 : Rescaling breakdown

A. Relation to Contributions: Measures the cost of a single rescaling operation and its per-stage decomposition against fleet size (C_1, C_2), on both CPU and GPU fleets (C_4), producing Figure 3. The same measurements supply the no-restart bars of the mode comparison in Figure 4, whose kill-and-restart bars are the published measurements of prior work.

B. Expected Results: A shrink costs 8–15 ms and an expansion 5–12 ms on CPU fleets of 4 to 32 instances, with GPU fleets within a few milliseconds of the CPU case; shrink is dominated by endpoint reconciliation, which is nearly flat in fleet size, and expansion by runtime initialization on the newcomer. The first rescaling operation of a run is several times more expensive than the rest, so the minimum over trials is reported.

C. Expected Reproduction Time (in minutes): Setup 60 per platform (image preparation); execution 60 (CPU) and 60 (GPU) for the full 4/8/16/32 sweep at three trials; analysis 5.

D. Artifact Setup:

Hardware: EC2 fleets of up to 32 `c6gn.large` instances (CPU) and up to 32 `g4dn.xlarge` instances (GPU), launched into a single availability zone with a cluster placement group.

Software: Charm++ with the no-restart extensions, built as `ucx-linux-arm8-openpmix` (CPU) and `ucx-linux-x86_64-cuda-openpmix` (GPU); UCX; OpenPMIx; Python 3 with `boto3`, `asyncssh`, and `matplotlib`.

Datasets/Input: none; Jacobi2D (CPU) and Stencil2D (GPU) run a $16,384 \times 16,384$ grid in $2,048 \times 2,048$ blocks.

Installation & Deployment: see the charm-aws README. The benchmark must be relinked whenever the runtime is rebuilt, since the source and build trees hold independent binaries.

E. Artifact Evaluation: T_1 (`run_breakdown.py` launches a fleet per instance count, runs the application, and issues shrink and expand requests over CCS for each trial) $\rightarrow T_2$ (the runtime emits a per-stage timing block on PE 0 for each operation, captured in the run log) $\rightarrow T_3$ (`plot_breakdown.py` and `plot_mode_comparison_norestart.py` render Figures 3 and 4). The driver aborts if EC2 fulfils fewer instances than requested, rather than recording a smaller cluster under the requested label.

F. Artifact Analysis: The driver parses the timing blocks into CSV and JSON, classifying each operation as a shrink or an expansion from the preceding coordinator commit, and reports the fastest trial per point.

Artifact A_2 : Interruption runs

A. Relation to Contributions: Runs the application to completion under simultaneous spot interruptions (C_1, C_5) on both platforms (C_4), producing Figure 5. Each batch of interruptions is handled by one combined rescaling operation, the capacity-rebalanced path of Section V.

B. Expected Results: End-to-end overhead grows slowly with the number of simultaneous interruptions and remains well below one percent of runtime on CPUs and approximately one percent on GPUs at eight interruptions, with the residual overhead attributable to load balancing rather than to the rescaling mechanism.

C. Expected Reproduction Time (in minutes): Setup as A_1 ; execution 150 per platform for interruption counts 0/1/2/4/8; analysis 5.

D. Artifact Setup:

Hardware: 16 instances for the job plus idle replacement capacity, per platform, as in A_1 .

Software, Datasets/Input, Installation: as A_1 .

E. Artifact Evaluation: T_1 (`run_endtoend.py` launches the fleet and starts the application) $\rightarrow T_2$ (at a fixed fraction of the run, k newcomers are spawned on the replacement instances and a single client request removes the k victims and admits the k replacements) $\rightarrow T_3$ (the run completes and its total execution time is compared against the uninterrupted baseline).

F. Artifact Analysis: The driver extracts total execution time from the application's completion output and reports overhead against the $k = 0$ run.

Artifact A_3 : Frequency study

A. Relation to Contributions: Establishes how often a job can be rescaled before the overhead becomes significant (C_1 , C_5), producing Figure 6.

B. Expected Results: Slowdown is inversely proportional to the rescaling period. Overhead stays below one percent for periods of tens of seconds, roughly an order of magnitude shorter than the periods at which restart-based rescaling reaches the same threshold, and the residual per-operation cost is dominated by load balancing rather than by the mechanism.

C. Expected Reproduction Time (in minutes): Setup as A_1 ; execution 120 per platform for the period sweep; analysis 5.

D. Artifact Setup:

Hardware: 16 instances per platform, as in A_1 .

Software, Datasets/Input, Installation: as A_1 .

E. Artifact Evaluation: T_1 (`run_frequency.py` launches the fleet and starts the application) $\rightarrow T_2$ (a background task alternates shrink and expand requests at the configured period until the run completes; a newcomer is spawned before each expansion) $\rightarrow T_3$ (execution time and the number of committed operations are compared against a baseline run with no rescaling).

F. Artifact Analysis: The driver computes slowdown relative to the baseline and divides the additional time by the number of committed rescaling operations to recover the per-operation cost, rendered by `plot_frequency.py`.